

A project report on

E-COMMERCE ORDER MANAGEMENT SYSTEM

Submitted by

KUDIPUDI PARAMESWARARAO

[Batch Code: ABM-2005-46]



ATLYS WORKFORCE TECHNOLOGIES

TABLE OF CONTENTS

S.No	Description	Page Number
1	Abstract	1
2	Database Schema Design	2
3	Data Insertion	7
4	Data Retrieval and Reports	10
5	Stored Procedure and Functions	29
6	Triggers	33
7	Performance Optimization	35
8	User Authentication	37
9	Conclusion	43

1. ABSTRACT

The **E-Commerce Order Management System** is designed to efficiently manage and organize data related to online shopping activities. The project focuses on creating a structured relational database that stores and maintains information about users, products, orders, order items, and payments. The system enables seamless tracking of customer purchases, inventory management, and payment processing while ensuring data integrity through the use of primary keys, foreign keys, and relational constraints.

The database consists of five core entities: **Users**, which store customer details; **Products**, which maintain product information and inventory levels; **Orders**, which record customer purchase transactions; **Order_Items**, which establish the relationship between orders and products; and **Payments**, which track payment details and transaction statuses. These entities are interconnected through well-defined relationships that support efficient data retrieval and management.

The system also supports advanced database functionalities, including stored procedures, triggers, and transaction management, to enhance performance and reliability. This project provides a practical understanding of relational database design and management in an e-commerce environment. It serves as a foundation for developing scalable online retail applications and highlights the importance of efficient data organization and consistency in modern business systems.

2. DATABASE SCHEMA DESIGN

EcommerceDB is created and used for the E-Commerce Order Management System.

```
-- Database Creation
CREATE DATABASE EcommerceDB;

-- Using EcommerceDB
USE EcommerceDB;
```

The database consists of five core entities: **Users**, **Products**, **Orders**, **Order_Items** and **Payments**.

2.1 Users Table:

The **Users** table is designed to store the basic information of users who register in the system. It acts as the primary table for user management and authentication.

```
-- Create Users Table
CREATE TABLE users (
    user_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL
);
```

Column Name	Data Type	Constraint	Description
user_id	INT	PRIMARY KEY	Unique identifier for each user.
name	VARCHAR(100)	NOT NULL	Stores the user's full name.
email	VARCHAR(100)	UNIQUE, NOT NULL	Stores the user's email address. Each email must be unique.
password	VARCHAR(255)	NOT NULL	Stores the encrypted or hashed password of the user.

2.2 Products Table:

The Products table is designed to store information about all products available in the e-commerce system. It serves as the central repository for product details such as product name, price, and available stock quantity. This table enables inventory management and supports order processing by providing product-related information.

```
-- Create Products Table
CREATE TABLE products (
  product_id INT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  price DECIMAL(10,2) NOT NULL,
  stock INT NOT NULL
);
```

Column Name	Data Type	Constraint	Description
product_id	INT	PRIMARY KEY	Unique identifier for each product.
name	VARCHAR(100)	NOT NULL	Stores the name of the product.
price	DECIMAL(10,2)	NOT NULL	Stores the product price with up to 10 digits and 2 decimal places.
stock	INT	NOT NULL	Stores the quantity of products available in inventory.

2.3 Orders Table:

The **Orders** table is designed to store information about customer orders placed in the e-commerce system. Each record in this table represents a single order made by a user. The table establishes a relationship between users and their orders, enabling the system to track purchasing activities and maintain order history.

```
-- Create Orders Table
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    user_id INT NOT NULL,
    order_date DATE NOT NULL,
    CONSTRAINT fk_orders_users
        FOREIGN KEY (user_id)
        REFERENCES users(user_id)
);
```

Column Name	Data Type	Constraint	Description
order_id	INT	PRIMARY KEY	Unique identifier for each order.
user_id	INT	FOREIGN KEY, NOT NULL	References the user who placed the order.
order_date	DATE	NOT NULL	Stores the date when the order was created.

2.4 Order_Items Table:

The **Order_Items** table is designed to store detailed information about the products included in each customer order. It acts as an associative table between the **Orders** and **Products** tables, enabling a many-to-many relationship between orders and products. Each record in this table represents a specific product and its quantity within a particular order.

```
-- Create Order_Items Table
CREATE TABLE order_items (
    order_item_id INT PRIMARY KEY,
    order_id INT NOT NULL,
    product_id INT NOT NULL,
    quantity INT NOT NULL CHECK (quantity > 0),

    CONSTRAINT fk_order_items_orders
        FOREIGN KEY (order_id)
        REFERENCES orders(order_id),

    CONSTRAINT fk_order_items_products
        FOREIGN KEY (product_id)
        REFERENCES products(product_id)
);
```

Column Name	Data Type	Constraint	Description
order_item_id	INT	PRIMARY KEY	Unique identifier for each order item record.
order_id	INT	FOREIGN KEY, NOT NULL	References the order in which the product was purchased.
product_id	INT	FOREIGN KEY, NOT NULL	References the purchased product.
quantity	INT	NOT NULL, CHECK	Stores the quantity of the product ordered. Must be greater than zero.

2.5 Payments Table:

The **Payments** table is designed to store information related to payments made for customer orders in the e-commerce system. It records the payment amount, payment status, and the order associated with each payment transaction. This table plays a crucial role in managing financial transactions and tracking payment activities.

```
-- Create Payments Table
CREATE TABLE payments (
    payment_id INT PRIMARY KEY,
    order_id INT NOT NULL,
    amount DECIMAL(10,2) NOT NULL,
    status VARCHAR(20) NOT NULL
    CHECK (status IN ('Completed', 'Pending', 'Failed', 'Refunded')),

    CONSTRAINT fk_payments_orders
        FOREIGN KEY (order_id)
        REFERENCES orders(order_id)
);
```

Column Name	Data Type	Constraint	Description
payment_id	INT	PRIMARY KEY	Unique identifier for each payment transaction.
order_id	INT	FOREIGN KEY, NOT NULL	References the order associated with the payment.
amount	DECIMAL(10,2)	NOT NULL	Stores the payment amount.
status	VARCHAR(20)	NOT NULL, CHECK	Stores the current status of the payment transaction.

3. DATA INSERTION

Data insertion is the process of adding new records into database tables using the SQL INSERT INTO statement. In an e-commerce database, data must be inserted in a specific order to maintain referential integrity because several tables are connected through foreign key relationships.

3.1 Inserting Data into Users Table:

The Users table stores customer information.

```
-- Insert 30 Random Records into the users table
INSERT INTO users (user_id, name, email, password) VALUES
(1, 'Aarav Sharma', 'aarav.sharma@email.com', 'Pass@101'),
(2, 'Priya Reddy', 'priya.reddy@email.com', 'Pass@102'),
(3, 'Rahul Verma', 'rahul.verma@email.com', 'Pass@103'),
(4, 'Sneha Patel', 'sneha.patel@email.com', 'Pass@104'),
(5, 'Karan Gupta', 'karan.gupta@email.com', 'Pass@105'),
(6, 'Ananya Singh', 'ananya.singh@email.com', 'Pass@106'),
(7, 'Vikram Rao', 'vikram.rao@email.com', 'Pass@107'),
(8, 'Meera Nair', 'meera.nair@email.com', 'Pass@108'),
(9, 'Rohan Kumar', 'rohan.kumar@email.com', 'Pass@109'),
(10, 'Pooja Das', 'pooja.das@email.com', 'Pass@110');
```

3.2 Inserting Data into Products Table:

The Products table stores inventory information.

```
-- Inserting 50 records into Product Records
INSERT INTO products (product_id, name, price, stock) VALUES
(1, 'Wireless Mouse', 599.00, 120),
(2, 'Mechanical Keyboard', 2499.00, 45),
(3, 'USB-C Charger 65W', 1499.00, 80),
(4, 'Bluetooth Speaker', 1999.00, 60),
(5, 'Gaming Headset', 3299.00, 35),
(6, 'Laptop Stand', 899.00, 75),
(7, 'HD Webcam', 2199.00, 50),
(8, 'External SSD 500GB', 4299.00, 40),
(9, 'Portable HDD 1TB', 3799.00, 55),
(10, 'Wi-Fi Router', 2899.00, 30);
```

3.3 Inserting Data into Orders Table:

The Orders table records customer purchases.

```
-- Inserting 30 Sample Order Records
-- Assumes users table contains user_id values from 1 to 30
INSERT INTO orders (order_id, user_id, order_date) VALUES
(1, 1, '2025-01-05'),
(2, 2, '2025-01-08'),
(3, 3, '2025-01-10'),
(4, 1, '2025-01-15'),
(5, 4, '2025-01-18'),
(6, 5, '2025-01-20'),
(7, 2, '2025-01-25'),
(8, 6, '2025-02-01'),
(9, 7, '2025-02-03'),
(10, 3, '2025-02-05');
```

3.4 Inserting Data into Order_Items Table:

The Order_Items table stores products purchased in each order.

```
-- Insert 30 Sample Records
-- Assumes: orders table contains order_id values from 1 to 30
-- products table contains product_id values from 1 to 50
INSERT INTO order_items (order_item_id, order_id, product_id, quantity) VALUES
(1, 1, 5, 2),
(2, 1, 12, 1),
(3, 2, 8, 3),
(4, 3, 15, 1),
(5, 4, 20, 2),
(6, 5, 3, 4),
(7, 6, 25, 1),
(8, 7, 10, 2),
(9, 8, 18, 1),
(10, 9, 30, 3);
```

3.5 Inserting Data into Payments Table:

The Payments table stores transaction details.

```
-- Inserting 30 Sample Payment Records
-- Assumes order_id values from 1 to 30 exist in the orders table
INSERT INTO payments (payment_id, order_id, amount, status) VALUES
(1, 1, 6598.00, 'Completed'),
(2, 2, 4299.00, 'Completed'),
(3, 3, 499.00, 'Completed'),
(4, 4, 14998.00, 'Completed'),
(5, 5, 5996.00, 'Completed'),
(6, 6, 5996.00, 'Completed'),
(7, 7, 2899.00, 'Pending'),
(8, 8, 6199.00, 'Completed'),
(9, 9, 26997.00, 'Completed'),
(10, 10, 4398.00, 'Completed');
```

4. DATA RETRIEVAL AND REPORTS

4.1 Query to calculate the total number of orders placed by each customer in the e-commerce system.

```
-- 1. Total Orders Placed by Each Customer
SELECT
    u.user_id,
    u.name,
    COUNT(o.order_id) AS total_orders
FROM users u
JOIN orders o
    ON u.user_id = o.user_id
GROUP BY u.user_id, u.name LIMIT 15;
```

Output:

Result Grid				Filter Rows:	Export:	Wrap Cell Content:	Fetch rows:
	user_id	name	total_orders				
▶	1	Aarav Sharma	2				
	2	Priya Reddy	2				
	3	Rahul Verma	2				
	4	Sneha Patel	2				
	5	Karan Gupta	2				
	6	Ananya Singh	2				
	7	Vikram Rao	2				
	8	Meera Nair	2				
	9	Rohan Kumar	2				
	10	Pooja Das	2				
	11	Arjun Mehta	1				
	12	Kavya Iyer	1				
	13	Nikhil Joshi	1				
	14	Divya Kapoor	1				
	15	Siddharth Jain	1				

4.2 Query to calculate the total revenue earned by the e-commerce system from successfully completed payments. It sums all payment amounts recorded in the payments table where the payment status is marked as Completed.

```
-- 2. Total Revenue Generated
SELECT
    SUM(amount) AS total_revenue
FROM payments
WHERE status = 'Completed';
```

Output:

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	total_revenue			
▶	187301.00			

4.3 Query is used to calculate the total quantity sold for each product in the e-commerce system. It retrieves product details from the products table and combines them with sales data from the order_items table to determine how many units of each product have been sold.

```
-- 3. Product Wise Quantity Sold
SELECT
    p.product_id,
    p.name,
    SUM(oi.quantity) AS total_quantity_sold
FROM products p
JOIN order_items oi
    ON p.product_id = oi.product_id
GROUP BY p.product_id, p.name ORDER BY p.product_id;
```

Output:

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
product_id	name	total_quantity_sold	
2	Mechanical Keyboard	1	
3	USB-C Charger 65W	4	
5	Gaming Headset	2	
6	Laptop Stand	2	
7	HD Webcam	2	
8	External SSD 500GB	3	
9	Portable HDD 1TB	5	
10	Wi-Fi Router	2	
11	LED Desk Lamp	1	
12	Smartphone Tripod	1	
13	Noise Cancelling Ear...	2	
14	Monitor 24 Inch	2	
15	USB Hub 4-Port	1	
16	Power Bank 10000mAh	4	
18	Graphics Tablet	1	
19	Smart Watch	1	
20	Fitness Band	2	

4.4 Query to retrieve detailed information about customer orders by combining data from the Users and Orders tables. It displays the customer's name along with the corresponding order ID and order date.

```
-- 4. Customer Order Details
SELECT
    u.name,
    o.order_id,
    o.order_date
FROM users u
INNER JOIN orders o
ON u.user_id = o.user_id;
```

Output:

Result Grid  Filter Rows: Export:  Wrap Cell Content: 			
	name	order_id	order_date
▶	Aarav Sharma	1	2025-01-05
	Priya Reddy	2	2025-01-08
	Rahul Verma	3	2025-01-10
	Aarav Sharma	4	2025-01-15
	Sneha Patel	5	2025-01-18
	Karan Gupta	6	2025-01-20
	Priya Reddy	7	2025-01-25
	Ananya Singh	8	2025-02-01
	Vikram Rao	9	2025-02-03
	Rahul Verma	10	2025-02-05
	Meera Nair	11	2025-02-08
	Rohan Kumar	12	2025-02-12
	Sneha Patel	13	2025-02-15
	Pooja Das	14	2025-02-18
	Arjun Mehta	15	2025-02-20

4.5 Query to retrieve comprehensive order details by combining information from multiple tables in the e-commerce database. It provides a complete view of each order, including customer details, product information, quantity purchased, payment amount, and payment status.

```
-- 5. Complete Order Information
SELECT
    o.order_id,
    u.name,
    p.name AS product_name,
    oi.quantity,
    pay.amount,
    pay.status
FROM orders o
JOIN users u
    ON o.user_id = u.user_id
JOIN order_items oi
    ON o.order_id = oi.order_id
JOIN products p
    ON oi.product_id = p.product_id
JOIN payments pay
    ON o.order_id = pay.order_id LIMIT 15;
```

Output:

Result Grid						
		Filter Rows:	Export:	Wrap Cell Content:	Fetch rows:	
	order_id	name	product_name	quantity	amount	status
▶	1	Aarav Sharma	Gaming Headset	2	6598.00	Completed
	1	Aarav Sharma	Smartphone Tripod	1	6598.00	Completed
	2	Priya Reddy	External SSD 500GB	3	4299.00	Completed
	3	Rahul Verma	USB Hub 4-Port	1	499.00	Completed
	4	Aarav Sharma	Fitness Band	2	14998.00	Completed
	5	Sneha Patel	USB-C Charger 65W	4	5996.00	Completed
	6	Karan Gupta	Monitor Arm	1	5996.00	Completed
	7	Priya Reddy	Wi-Fi Router	2	2899.00	Pending
	8	Ananya Singh	Graphics Tablet	1	6199.00	Completed
	9	Vikram Rao	Action Camera	3	26997.00	Completed
	10	Rahul Verma	HD Webcam	2	4398.00	Completed
	11	Meera Nair	Wireless Presenter	1	799.00	Failed
	12	Rohan Kumar	Monitor 24 Inch	2	21998.00	Completed
	13	Sneha Patel	Phone Case	1	399.00	Completed
	14	Pooja Das	Portable HDD 1TB	5	14495.00	Completed

4.6 Query to identify customers who have placed multiple orders in the e-commerce system.
It retrieves the complete details of users whose order count is greater than one.

```
-- 6. Customers Who Placed More Than One Order
SELECT *
FROM users
WHERE user_id IN
(
    SELECT user_id
    FROM orders
    GROUP BY user_id
    HAVING COUNT(order_id) > 1
);
```

Output:

Result Grid				
Filter Rows:				
Edit: 				
Export/Import:  				
Wrap Cell Content: 				
	user_id	name	email	password
▶	1	Aarav Sharma	aarav.sharma@email.com	Pass@101
	2	Priya Reddy	priya.reddy@email.com	Pass@102
	3	Rahul Verma	rahul.verma@email.com	Pass@103
	4	Sneha Patel	sneha.patel@email.com	Pass@104
	5	Karan Gupta	karan.gupta@email.com	Pass@105
	6	Ananya Singh	ananya.singh@email.com	Pass@106
	7	Vikram Rao	vikram.rao@email.com	Pass@107
	8	Meera Nair	meera.nair@email.com	Pass@108
	9	Rohan Kumar	rohan.kumar@email.com	Pass@109
	10	Pooja Das	pooja.das@email.com	Pass@110
*	NULL	NULL	NULL	NULL

4.7 Query to identify products that have never been purchased by any customer. It retrieves all product records from the products table whose product_id does not appear in the order_items table.

```
-- 7. Products Never Ordered
SELECT *
FROM products
WHERE product_id NOT IN
(
    SELECT DISTINCT product_id
    FROM order_items
);
```

Output:

Result Grid				
		Filter Rows:		
		Edit:		
		Export/Import:		
	product_id	name	price	stock
▶	1	Wireless Mouse	599.00	120
	4	Bluetooth Speaker	1999.00	60
	17	USB Microphone	2599.00	28
	23	Ethernet Cable 5m	249.00	200
	26	Desk Organizer	599.00	110
	29	Wireless Charger	1699.00	44
	31	Photo Printer	12499.00	8
	32	Ink Cartridge Black	1299.00	95
	34	Desk Mat XL	799.00	66
	36	Tempered Glass	299.00	170
	37	Car Charger Dual ...	699.00	88
	39	Smart Plug	1199.00	53
	41	Portable Projector	15999.00	6
	42	Laser Pointer	499.00	57
	43	Cable Manageme...	349.00	145
	44	Laptop Sleeve 15 ...	899.00	73
	46	USB Flash Drive 2...	1699.00	64

4.8 Query to identify customers who have spent the most money on completed purchases in the e-commerce system. It calculates the total amount spent by each customer using payment records and then ranks customers in descending order based on their spending.

```
-- Common Table Expressions (CTE)
-- 8. Top Customers by Spending
WITH CustomerSpending AS
(
    SELECT
        o.user_id,
        SUM(p.amount) AS total_spent
    FROM orders o
    JOIN payments p
    ON o.order_id = p.order_id
    WHERE p.status = 'Completed'
    GROUP BY o.user_id
)
SELECT *
FROM CustomerSpending
ORDER BY total_spent DESC;
```

Output:

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	user_id	total_spent			
▶	9	39995.00			
	8	29998.00			
	7	29995.00			
	1	21596.00			
	10	14495.00			
	5	7794.00			
	17	7499.00			
	4	6395.00			
	6	6199.00			
	14	5596.00			
	3	4897.00			
	2	4299.00			
	12	2997.00			
	15	2499.00			
	20	1599.00			
	13	1099.00			
	18	349.00			

4.9 Query to calculate a running total of payment amounts using a Recursive Common Table Expression (Recursive CTE). A running total continuously adds the current payment amount to the sum of all previous payments

```
WITH RECURSIVE RevenueSeries AS
(
    SELECT
        payment_id,
        amount,
        amount AS running_total
    FROM payments
    WHERE payment_id = 1

    UNION ALL

    SELECT
        p.payment_id,
        p.amount,
        rs.running_total + p.amount
    FROM payments p
    JOIN RevenueSeries rs
        ON p.payment_id = rs.payment_id + 1
)
SELECT *
FROM RevenueSeries;
```

Output:

Result Grid Filter Rows: Export: Wrap Cell Content: FA			
	payment_id	amount	running_total
▶	1	6598.00	6598.00
	2	4299.00	10897.00
	3	499.00	11396.00
	4	14998.00	26394.00
	5	5996.00	32390.00
	6	5996.00	38386.00
	7	2899.00	41285.00
	8	6199.00	47484.00
	9	26997.00	74481.00
	10	4398.00	78879.00
	11	799.00	79678.00
	12	21998.00	101676.00
	13	399.00	102075.00
	14	14495.00	116570.00
	15	15999.00	132569.00
	16	1798.00	134367.00
	17	2997.00	137364.00

4.10 Query to identify and rank products based on the total quantity sold in the e-commerce system. It calculates the total units sold for each product and assigns a ranking using the SQL RANK() Window Function.

```
-- Window Functions
-- 10. Top Selling Products Ranking
SELECT
    p.product_id,
    p.name,
    SUM(oi.quantity) AS total_sold,
    RANK() OVER
    (
        ORDER BY SUM(oi.quantity) DESC
    ) AS ranking
FROM products p
JOIN order_items oi
ON p.product_id = oi.product_id
GROUP BY p.product_id, p.name;
```

Output:

Result Grid					Filter Rows:		Export:	Wrap Cell Content:
	product_id	name	total_sold	ranking				
▶	9	Portable HDD 1TB	5	1				
	3	USB-C Charger 65W	4	2				
	16	Power Bank 10000mAh	4	2				
	8	External SSD 500GB	3	4				
	27	Cooling Pad	3	4				
	28	Surge Protector	3	4				
	30	Action Camera	3	4				
	38	Bluetooth Receiver	3	4				
	5	Gaming Headset	2	9				
	6	Laptop Stand	2	9				
	7	HD Webcam	2	9				
	10	Wi-Fi Router	2	9				
	13	Noise Cancelling Earbuds	2	9				
	14	Monitor 24 Inch	2	9				
	20	Fitness Band	2	9				
	33	Ink Cartridge Color	2	9				
	45	USB Flash Drive 128GB	2	9				

4.11 Query to identify and rank products based on the total quantity sold in the e-commerce system. It calculates the total number of units sold for each product and assigns a rank using the DENSE_RANK() Window Function.

Unlike the RANK() function, DENSE_RANK() does not skip rank numbers when multiple products have the same sales quantity. Products with equal sales receive the same rank, and the next rank is assigned sequentially.

```
-- 11. Top Selling Products Ranking usnig DENSE_RANK()
WITH product_sales AS
(
    SELECT
        p.product_id,
        p.name,
        SUM(oi.quantity) AS total_sold
    FROM products p
    JOIN order_items oi
        ON p.product_id = oi.product_id
    GROUP BY p.product_id, p.name
)
SELECT
    product_id,
    name,
    total_sold,
    DENSE_RANK() OVER
    (
        ORDER BY total_sold DESC
    ) AS ranking
FROM product_sales;
```

Output:

Result Grid Filter Rows: Export: Wrap Cell Content: IA				
	product_id	name	total_sold	ranking
▶	9	Portable HDD 1TB	5	1
	3	USB-C Charger 65W	4	2
	16	Power Bank 10000mAh	4	2
	8	External SSD 500GB	3	3
	30	Action Camera	3	3
	28	Surge Protector	3	3
	27	Cooling Pad	3	3
	38	Bluetooth Receiver	3	3
	5	Gaming Headset	2	4
	20	Fitness Band	2	4
	10	Wi-Fi Router	2	4
	7	HD Webcam	2	4
	14	Monitor 24 Inch	2	4
	6	Laptop Stand	2	4
	45	USB Flash Drive 128GB	2	4
	33	Ink Cartridge Color	2	4
	50	Portable Monitor 15.6 ...	2	4

4.12 Query to display each customer's order history along with the date of their previous order. It utilizes the LAG() Window Function to access data from the previous row within the same customer group.

```
-- 12. Previous Order Date Per Customer
SELECT
    user_id,
    order_id,
    order_date,

    LAG(order_date)
    OVER
    (
        PARTITION BY user_id
        ORDER BY order_date
    ) AS previous_order

FROM orders;
```

Output:

	user_id	order_id	order_date	previous_order
▶	1	1	2025-01-05	NULL
	1	4	2025-01-15	2025-01-05
	2	2	2025-01-08	NULL
	2	7	2025-01-25	2025-01-08
	3	3	2025-01-10	NULL
	3	10	2025-02-05	2025-01-10
	4	5	2025-01-18	NULL
	4	13	2025-02-15	2025-01-18
	5	6	2025-01-20	NULL
	5	16	2025-02-24	2025-01-20
	6	8	2025-02-01	NULL
	6	19	2025-03-08	2025-02-01
	7	9	2025-02-03	NULL
	7	22	2025-03-18	2025-02-03
	8	11	2025-02-08	NULL
	8	25	2025-03-25	2025-02-08

4.13 Query to display each customer's order history along with the date of their next order. It utilizes the LEAD() Window Function to access data from the following row within the same customer group.

```
-- 13. Next Order Date Per Customer
SELECT
    user_id,
    order_id,
    order_date,

    LEAD(order_date)
    OVER
    (
        PARTITION BY user_id
        ORDER BY order_date
    ) AS next_order

FROM orders;
```

Output:

Result Grid Filter Rows: Export: Wrap Cell Content:				
	user_id	order_id	order_date	next_order
▶	1	1	2025-01-05	2025-01-15
	1	4	2025-01-15	NULL
	2	2	2025-01-08	2025-01-25
	2	7	2025-01-25	NULL
	3	3	2025-01-10	2025-02-05
	3	10	2025-02-05	NULL
	4	5	2025-01-18	2025-02-15
	4	13	2025-02-15	NULL
	5	6	2025-01-20	2025-02-24
	5	16	2025-02-24	NULL
	6	8	2025-02-01	2025-03-08
	6	19	2025-03-08	NULL
	7	9	2025-02-03	2025-03-18
	7	22	2025-03-18	NULL
	8	11	2025-02-08	2025-03-25
	8	25	2025-03-25	NULL

4.14 Query to calculate the running total (cumulative revenue) of payment amounts for each payment status category. It uses the SUM() Window Function to continuously add payment amounts within each status group while maintaining individual row details.

```
-- 14. Running Total Revenue by Payment Status using SUM()
SELECT
    payment_id,
    status,
    amount,

    SUM(amount)
    OVER
    (
        PARTITION BY status
        ORDER BY payment_id
    ) AS running_total

FROM payments LIMIT 10;
```

Output:

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	payment_id	status	amount	running_total
▶	1	Completed	6598.00	6598.00
	2	Completed	4299.00	10897.00
	3	Completed	499.00	11396.00
	4	Completed	14998.00	26394.00
	5	Completed	5996.00	32390.00
	6	Completed	5996.00	38386.00
	8	Completed	6199.00	44585.00
	9	Completed	26997.00	71582.00
	10	Completed	4398.00	75980.00
	12	Completed	21998.00	97978.00

4.15 Query to calculate the running average (cumulative average) of payment amounts using the SQL AVG() Window Function. The running average shows how the average payment amount changes as each new payment record is processed.

```
-- 15. Running Average Payment
SELECT
    payment_id,
    amount,

    AVG(amount)
    OVER
    (
        ORDER BY payment_id
    ) AS running_average

FROM payments;
```

Output:

Result Grid  Filter Rows: Export:  Wrap Cell Content: 			
	payment_id	amount	running_average
▶	1	6598.00	6598.000000
	2	4299.00	5448.500000
	3	499.00	3798.666667
	4	14998.00	6598.500000
	5	5996.00	6478.000000
	6	5996.00	6397.666667
	7	2899.00	5897.857143
	8	6199.00	5935.500000
	9	26997.00	8275.666667
	10	4398.00	7887.900000

4.16 Query uses the NTILE() Window Function to divide payment records into a specified number of groups (buckets) based on the payment amount within each payment status category.

In this example, the payments are divided into 3 buckets separately for each payment status such as Completed, Pending, Failed, and Refunded. The highest payment amounts receive the lowest bucket numbers because the records are sorted in descending order.

```
-- 16. NTILE with PARTITION BY Status
SELECT
    payment_id,
    status,
    amount,

    NTILE(3)
    OVER
    (
        PARTITION BY status
        ORDER BY amount DESC
    ) AS payment_bucket

FROM payments;
```

Output:

	payment_id	status	amount	payment_bucket
▶	25	Completed	29998.00	1
	9	Completed	26997.00	1
	12	Completed	21998.00	1
	28	Completed	17997.00	1
	4	Completed	14998.00	1
	14	Completed	14495.00	1
	24	Completed	7499.00	1
	1	Completed	6598.00	1
	8	Completed	6199.00	2
	5	Completed	5996.00	2
	6	Completed	5996.00	2
	20	Completed	5596.00	2
	10	Completed	4398.00	2

4.17 Query to generate a Monthly Revenue Report by calculating the total revenue earned from completed payments for each month and year. It combines data from the Orders and Payments tables to determine how much revenue was generated during specific time periods.

```
-- Advanced Analytics Report
-- 17. Monthly Revenue Report
SELECT
    YEAR(o.order_date) AS year,
    MONTH(o.order_date) AS month,
    SUM(p.amount) AS revenue
FROM orders o
JOIN payments p
ON o.order_id = p.order_id
WHERE p.status = 'Completed'
GROUP BY
    YEAR(o.order_date),
    MONTH(o.order_date)
ORDER BY year, month;
```

Output:

Result Grid				Filter Rows:	Export:	Wrap Cell Content:
	year	month	revenue			
▶	2025	1	38386.00			
	2025	2	76284.00			
	2025	3	53035.00			
	2025	4	19596.00			

4.18 Query to identify the top 5 customers who have spent the highest amount on purchases. It calculates the total amount spent by each customer using completed payments and ranks them in descending order of spending.

```
-- 18. Top 5 Customers
SELECT
    u.user_id,
    u.name,
    SUM(pay.amount) AS total_spent
FROM users u
JOIN orders o
    ON u.user_id = o.user_id
JOIN payments pay
    ON o.order_id = pay.order_id
WHERE pay.status = 'Completed'
GROUP BY u.user_id, u.name
ORDER BY total_spent DESC
LIMIT 5;
```

Output:

Result Grid	Filter Rows:	Export:	Wrap Cell Content:	Fetch rows:
user_id	name	total_spent		
9	Rohan Kumar	39995.00		
8	Meera Nair	29998.00		
7	Vikram Rao	29995.00		
1	Aarav Sharma	21596.00		
10	Pooja Das	14495.00		

4.19 Query to retrieve all products with a stock quantity less than 20 units.

```
-- 19. Low Stock Products
SELECT
    product_id,
    name,
    stock
FROM products
WHERE stock < 20;
```

Output:

Result Grid			
Filter Rows:			
Edit:    Export/Import:   Wrap Cell Content: 			
	product_id	name	stock
▶	14	Monitor 24 Inch	18
	18	Graphics Tablet	12
	25	Monitor Arm	16
	30	Action Camera	10
	31	Photo Printer	8
	40	Mini UPS Router Backup	19
	41	Portable Projector	6
	47	VR Headset	4
	48	Smart Home Hub	14
	49	NAS Enclosure 2-Bay	7
	50	Portable Monitor 15.6 ...	11
*	NULL	NULL	NULL

5. STORED PROCEDURE AND FUNCTIONS

5.1 STORED PROCEDURE:

A Stored Procedure in MySQL is a named collection of one or more SQL statements that is stored in the database and executed as a single unit. It is used to perform specific tasks such as data retrieval, data manipulation, validation, reporting, and business logic processing.

Example: 1. The GetCustomerOrders stored procedure is designed to retrieve all orders placed by a specific customer. The customer is identified using a user ID passed as an input parameter. This procedure allows applications to fetch customer order information without repeatedly writing the same SQL query.

```
-- Stored Procedure: Get Customer Orders
DELIMITER $$

CREATE PROCEDURE GetCustomerOrders(IN p_user_id INT)
BEGIN
    SELECT
        o.order_id,
        o.order_date
    FROM orders o
    WHERE o.user_id = p_user_id;
END $$

DELIMITER ;
```

Execution:

```
-- Execute
CALL GetCustomerOrders(1);
```

Output:

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
	order_id	order_date	
▶	1	2025-01-05	
	4	2025-01-15	

Example: 2. The ProductSalesReport stored procedure is designed to generate a report showing the total quantity sold for each product. It retrieves product information from the products table and sales data from the order_items table, then calculates the total number of units sold for every product.

```
-- Stored Procedure: Product Sales Report
DELIMITER //

CREATE PROCEDURE ProductSalesReport()
BEGIN

    SELECT
        p.product_id,
        p.name,
        SUM(oi.quantity) AS total_sold
    FROM products p
    JOIN order_items oi
        ON p.product_id = oi.product_id
    GROUP BY p.product_id, p.name;

END //

DELIMITER ;
```

Execution:

```
-- Execute
CALL ProductSalesReport();
```

Output:

Result Grid Filter Rows: Export: Wrap Cell Content:			
	product_id	name	total_sold
▶	5	Gaming Headset	2
	12	Smartphone Tripod	1
	8	External SSD 500GB	3
	15	USB Hub 4-Port	1
	20	Fitness Band	2
	3	USB-C Charger 65W	4
	25	Monitor Arm	1
	10	Wi-Fi Router	2
	18	Graphics Tablet	1
	30	Action Camera	3
	7	HD Webcam	2
	22	Wireless Presenter	1
	14	Monitor 24 Inch	2
	35	Phone Case	1
	9	Portable HDD 1TB	5
	40	Mini UPS Router Ba...	1
	6	Laptop Stand	2

5.2 FUNCTION:

A Function in MySQL is a stored database object that performs a specific task, accepts input parameters, processes data, and returns a single value. Functions are used to encapsulate reusable business logic and can be called directly within SQL statements such as SELECT, WHERE, ORDER BY, and HAVING.

Unlike stored procedures, a function must always return a value using the RETURN statement.

Example: 1. The GetCustomerTotalSpent function calculates the total amount spent by a specific customer based on all completed payments associated with their orders. It accepts a customer ID as an input parameter and returns the cumulative amount paid by that customer.

```
-- Function: Total Amount Paid by Customer
DELIMITER //

CREATE FUNCTION GetCustomerTotalSpent
(
    customer_id INT
)
RETURNS DECIMAL(10,2)

DETERMINISTIC

BEGIN

    DECLARE total DECIMAL(10,2);

    SELECT SUM(pay.amount)
    INTO total
    FROM orders o
    JOIN payments pay
        ON o.order_id = pay.order_id
    WHERE o.user_id = customer_id
    AND pay.status = 'Completed';

    RETURN IFNULL(total,0);

END //

DELIMITER ;
```

Execution:

```
-- Execute  
SELECT GetCustomerTotalSpent(1);
```

Output:

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	GetCustomerTotalSpent(1)			
▶	21596.00			

6. TRIGGERS

A Trigger in MySQL is a special type of stored database object that automatically executes (fires) when a specified event occurs on a table. Triggers are associated with INSERT, UPDATE, or DELETE operations and are used to enforce business rules, maintain data integrity, audit changes, and automate database tasks.

Unlike stored procedures and functions, triggers are executed automatically by MySQL and do not need to be called explicitly.

Example: 1. The `trg_reduce_stock` trigger is designed to automatically reduce the available stock of a product whenever a new order item is added to the `order_items` table. By automatically updating stock quantities after an order is placed, the trigger helps maintain inventory integrity and eliminates the need for manual stock adjustments.

```
-- Trigger: Reduce Product Stock After Order
```

```
DELIMITER //
```

```
CREATE TRIGGER trg_reduce_stock
```

```
AFTER INSERT
```

```
ON order_items
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    UPDATE products
```

```
    SET stock = stock - NEW.quantity
```

```
    WHERE product_id = NEW.product_id;
```

```
END //
```

```
DELIMITER ;
```

Example: 2. The `trg_check_stock` trigger is designed to prevent customers from placing orders that exceed the available inventory. Before a new order item is inserted into the `order_items` table, the trigger checks whether sufficient stock exists for the selected product.

If the requested quantity is greater than the available stock, the trigger stops the transaction and displays an error message indicating that there is insufficient stock.

```
-- Trigger: Prevent Negative Stock
DELIMITER //

CREATE TRIGGER trg_check_stock
BEFORE INSERT
ON order_items

FOR EACH ROW

BEGIN
    DECLARE available_stock INT;

    SELECT stock
    INTO available_stock
    FROM products
    WHERE product_id = NEW.product_id;

    IF available_stock < NEW.quantity THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Insufficient Stock';
    END IF;
END //

DELIMITER ;
```

7. PERFORMANCE OPTIMIZATION

Performance Optimization is the process of improving the speed, efficiency, and resource utilization of a database system. It involves optimizing queries, database design, indexing, server configuration, and application interactions to reduce execution time and improve overall database performance.

The primary goal is to ensure that database operations such as data retrieval, insertion, updating, and deletion are performed as quickly and efficiently as possible. Common Performance Optimization Techniques are:

1. Indexing
2. Optimize SQL Queries
3. Limit Returned Rows

INDEXING:

Indexing is a database optimization technique used to improve the speed of data retrieval operations. An index is a special data structure that stores references to table rows and allows MySQL to locate data quickly without scanning every row in a table.

- Without an index, MySQL performs a Full Table Scan, meaning it checks every row in the table to find matching records.
- With an index, MySQL can directly locate the required rows, significantly reducing query execution time.

Example: Without Index

```
SELECT *  
FROM users  
WHERE email = 'john@example.com';
```

MySQL may need to search every record in the users table.

Example: With an Index

```
CREATE INDEX idx_email
ON users(email);

SELECT *
FROM users
WHERE email = 'john@example.com';
```

Now MySQL can quickly locate the matching email address.

Indexes on foreign keys speed up joins.

```
-- Performance Optimization
-- Indexing: Index on Foreign Keys
CREATE INDEX idx_orders_user
ON orders(user_id);

CREATE INDEX idx_orderitems_order
ON order_items(order_id);

CREATE INDEX idx_orderitems_product
ON order_items(product_id);

CREATE INDEX idx_payments_order
ON payments(order_id);

-- Index on Payment Status
CREATE INDEX idx_payment_status
ON payments(status);
```

8. USER AUTHENTICATION

User Authentication is the process of verifying the identity of a user before granting access to a system, application, or database. It ensures that only authorized users can access protected resources by validating credentials such as a username and password.

8.1 User Authentication Table:

The user_login table is designed to manage and store login credentials and authentication-related information for users in the system. This table is linked to the users table through a foreign key relationship, allowing each user account to have associated authentication details while maintaining data integrity and normalization.

```
-- User Authentication Table
CREATE TABLE user_login
(
    login_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT,
    username VARCHAR(50) UNIQUE,
    password_hash VARCHAR(255),
    last_login DATETIME,

    FOREIGN KEY(user_id)
    REFERENCES users(user_id)
);
```

Column Name	Data Type	Constraint	Description
login_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique login record identifier
user_id	INT	FOREIGN KEY	References the user in the users table

Column Name	Data Type	Constraint	Description
username	VARCHAR(50)	UNIQUE	Unique username used for login
password_hash	VARCHAR(255)	-	Stores encrypted password
last_login	DATETIME	-	Records the user's most recent login

8.2 Inserting Data into user_login Table:

The user_login table stores login details of the users.

```
-- Inserting 20 random records into user_login
INSERT INTO user_login
(
    user_id,
    username,
    password_hash,
    last_login
)
VALUES
(1, 'aarav01', SHA2('Password@101',256), '2026-06-01 09:15:22'),
(2, 'priya02', SHA2('Password@102',256), '2026-06-02 10:30:15'),
(3, 'rahul03', SHA2('Password@103',256), '2026-06-03 11:45:08'),
(4, 'sneha04', SHA2('Password@104',256), '2026-06-04 08:20:35'),
(5, 'karan05', SHA2('Password@105',256), '2026-06-05 14:12:18'),
(6, 'ananya06', SHA2('Password@106',256), '2026-06-06 16:05:40'),
(7, 'vikram07', SHA2('Password@107',256), '2026-06-07 09:55:10'),
(8, 'meera08', SHA2('Password@108',256), '2026-06-08 13:22:47'),
(9, 'rohan09', SHA2('Password@109',256), '2026-06-09 17:40:05'),
(10, 'pooja10', SHA2('Password@110',256), '2026-06-10 10:18:32');
```


8.3 Queries:

8.3.1 Query to retrieve complete login information of users by combining data from the users table and the user_login table. It displays user details along with their login credentials and the date/time of their most recent login.

```
-- 1. View User Login Details
SELECT
    u.user_id,
    u.name,
    u.email,
    ul.username,
    ul.last_login
FROM users u
JOIN user_login ul
    ON u.user_id = ul.user_id;
```

Output:

Result Grid   Filter Rows: Export:  Wrap Cell Content: 					
	user_id	name	email	username	last_login
▶	1	Aarav Sharma	aarav.sharma@email.com	aarav01	2026-06-01 09:15:22
	2	Priya Reddy	priya.reddy@email.com	priya02	2026-06-02 10:30:15
	3	Rahul Verma	rahul.verma@email.com	rahul03	2026-06-03 11:45:08
	4	Sneha Patel	sneha.patel@email.com	sneha04	2026-06-04 08:20:35
	5	Karan Gupta	karan.gupta@email.com	karan05	2026-06-05 14:12:18
	6	Ananya Singh	ananya.singh@email.com	ananya06	2026-06-06 16:05:40
	7	Vikram Rao	vikram.rao@email.com	vikram07	2026-06-07 09:55:10
	8	Meera Nair	meera.nair@email.com	meera08	2026-06-08 13:22:47
	9	Rohan Kumar	rohan.kumar@email.com	rohan09	2026-06-09 17:40:05
	10	Pooja Das	pooja.das@email.com	pooja10	2026-06-10 10:18:32
	11	Arjun Mehta	arjun.mehta@email.com	arjun11	2026-06-11 12:45:56
	12	Kavya Iyer	kavya.iyer@email.com	kavya12	2026-06-12 15:30:14
	13	Nikhil Joshi	nikhil.joshi@email.com	nikhil13	2026-06-13 09:10:45
	14	Divya Kapoor	divya.kapoor@email.com	divya14	2026-06-14 18:05:27
	15	Siddharth Jain	siddharth.jain@email.com	siddharth15	2026-06-15 11:20:38

8.3.2 Query to identify users who have registered in the system but have never logged in. It retrieves user information from the users table and compares it with login records in the user_login table. Users whose last_login value is NULL are considered as inactive or have never accessed the system.

```
-- 2. Find Users Who Never Logged In
SELECT
    u.user_id,
    u.name
FROM users u
LEFT JOIN user_login ul
    ON u.user_id = ul.user_id
WHERE ul.last_login IS NULL;
```

Output:

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	user_id	name		
▶	21	Amit Tiwari		
	22	Nisha Bansal		
	23	Deepak Saxena		
	24	Tanvi Arora		
	25	Harsh Vardhan		
	26	Ishita Malhotra		
	27	Yash Thakur		
	28	Simran Kaur		
	29	Varun Sehgal		
	30	Ayesha Khan		

8.3.3 Query to retrieve the usernames and their most recent login timestamps from the user_login table and displays them in descending order of login time. The result shows the users who logged into the system most recently at the top of the list.

```
-- 3. Last Logged-In Users
SELECT
    username,
    last_login
FROM user_login
ORDER BY last_login DESC;
```

Output:

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	username	last_login			
▶	shreya20	2026-06-20 10:50:03			
	manish19	2026-06-19 19:15:44			
	ritika18	2026-06-18 08:35:52			
	aditya17	2026-06-17 16:40:29			
	neha16	2026-06-16 14:55:11			
	siddharth15	2026-06-15 11:20:38			
	divya14	2026-06-14 18:05:27			
	nikhil13	2026-06-13 09:10:45			

8.3.4 Query to rank users based on their most recent login activity using the ROW_NUMBER() window function. Users who logged in most recently receive the highest rank (Rank 1), while users with older login dates receive lower rankings.

```
-- 4. Login Activity Ranking Using a window function:
SELECT
    username,
    last_login,

    ROW_NUMBER() OVER
    (
        ORDER BY last_login DESC
    ) AS login_rank

FROM user_login;
```

Output:

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
username	last_login	login_rank	
shreya20	2026-06-20 10:50:03	1	
manish19	2026-06-19 19:15:44	2	
ritika18	2026-06-18 08:35:52	3	
aditya17	2026-06-17 16:40:29	4	
neha16	2026-06-16 14:55:11	5	
siddharth15	2026-06-15 11:20:38	6	
divya14	2026-06-14 18:05:27	7	
nikhil13	2026-06-13 09:10:45	8	
kavya12	2026-06-12 15:30:14	9	
arjun11	2026-06-11 12:45:56	10	

9. CONCLUSION

The E-Commerce Order Management System project successfully demonstrates the design and implementation of a relational database for managing online shopping operations. The system effectively stores and manages information related to customers, products, orders, order items, payments, and user authentication while maintaining data integrity through the use of primary keys, foreign keys, and relational constraints.

The developed system can be further extended with web-based interfaces, real-time analytics, advanced security mechanisms, and cloud-based deployment to meet the growing demands of modern online retail platforms.